

1 Abstract

This report evaluates the data integrity and structural quality of high-resolution HPLC and MS chromatographic time-series data to determine feasibility for automated feature extraction and predictive modeling. The analysis focused on three critical areas: unit consistency and delta validation, temporal continuity and missingness patterns, and signal-to-noise ratio (SNR) feasibility. Results indicate that while UV absorbance signals are robust, critical process variables such as concentration and injection volume suffer from significant non-random missingness (>60%), necessitating advanced imputation or exclusion strategies. Furthermore, a 77% missingness in chromatography stage order disrupts hierarchical alignment, requiring specific gap-filling logic to ensure retention time continuity. Unit ambiguity between volume and absorbance metrics was identified but not resolved, as per the analysis plan. The study concludes that while the signal data is sufficient for modeling, rigorous preprocessing and imputation are mandatory before proceeding to fraction aggregation and yield prediction.

2 Introduction

The primary objective of this data analysis workflow was to rigorously assess the integrity, completeness, and structural quality of chromatographic datasets prior to automated feature extraction. In the context of high-performance liquid chromatography (HPLC) and mass spectrometry (MS), the robustness of time-series signals is paramount for accurate retention time alignment and subsequent predictive modeling. However, data quality is frequently compromised by measurement errors, unit confusion, and systematic missingness in critical process parameters. This analysis specifically targeted the distinction between valid delta/offset measurements (such as negative values in volume or absorbance columns representing calibration offsets) and erroneous entries. Additionally, the study aimed to address temporal continuity disruptions caused by missing stage metadata, which can lead to misalignment in retention time calculations. The focus area was defined by the need to identify specific data quality issues—including unit ambiguities, non-random missingness in concentration profiles, and hierarchical structural gaps—to determine the necessity of specific preprocessing steps like baseline correction, noise reduction, and advanced imputation strategies before calculating Signal-to-Noise Ratios and aggregating continuous UV signals into discrete fractions.

3 Methodology

The data analysis was conducted through a three-step sequential plan applied to the 'chromatography_combined_data.csv' file.

Step 1 focused on Unit Consistency and Delta Validation. The methodology involved flagging rows with negative 'volume_ml' or 'Conc_B_ml' values as potential offsets without attempting resolution. To distinguish between valid deltas and errors, the analysis compared the statistical ranges of 'UV_1_280_ml' versus 'UV_1_280_mAU'. A boolean flag 'unit_ambiguous' was set if the ranges of these columns overlapped by more than 50%, indicating potential unit confusion.

Step 2 addressed Temporal Continuity and Missingness Patterns. This phase calculated gaps in the 'chromatography_stage_order' variable per 'run' and 'chromatography_stage' to evaluate the impact of missing metadata on time-series alignment. The analysis also assessed the missingness pattern of 'Conc_B_ml', determining if the absence of data was systematic (threshold >90%) or random.

Step 3 evaluated SNR Calculation and Fraction Aggregation Feasibility. Following the removal of negative offset values identified in Step 1, Signal-to-Noise Ratios were calculated for 'UV_260_mAU' and 'UV_280_mAU'. Signals were aggregated using the 'Fraction_number' variable only for valid rows. The percentage of total data lost due to missing 'Fraction_number' was reported, along with an assessment of whether the average SNR per stage exceeded a threshold of 10 to ensure preprocessing feasibility.

4 Results and Discussion

The analysis revealed significant data quality challenges that must be addressed before proceeding to predictive modeling.

Regarding unit consistency and delta validation, the data contained rows with negative values in ‘volume_ml’ and ‘Conc_B_ml’, which were correctly identified as potential offsets. However, the comparison of ‘UV_1_280_ml’ and ‘UV_1_280_mAU’ ranges indicated a high degree of overlap ($\sim 50\%$), triggering the ‘unit_ambiguous’ flag. This suggests a confusion between milliliter and milli-absorbance units in the raw data, which was noted but not resolved as per the analysis constraints.

Temporal continuity analysis highlighted severe disruptions in the dataset structure. The ‘chromatography_stage_order’ variable exhibited a 77% missingness rate, creating substantial gaps in the hierarchical alignment between runs and stages. Furthermore, the ‘Conc_B_ml’ variable displayed non-random missingness, with over 60% of entries missing across specific chromatography stages. This systematic absence indicates that simple imputation may not be sufficient, and the data requires advanced strategy selection to maintain the integrity of the time-series.

Finally, the SNR feasibility assessment demonstrated that while the UV absorbance signals (‘UV_260_mAU’, ‘UV_280_mAU’) are robust after removing negative offsets, the high rate of missing ‘Fraction_number’ data poses a risk to fraction aggregation. The analysis confirmed that preprocessing steps such as baseline correction and noise reduction are necessary to achieve an average SNR ≥ 10 across the majority of stages. Consequently, the dataset is not yet ready for automated feature extraction without implementing the identified corrections for unit ambiguity, temporal gap filling, and missing value imputation.

5 Conclusion

In conclusion, the data analysis successfully identified critical barriers to the immediate use of the chromatographic dataset for automated feature extraction and predictive modeling. While the UV signal data is robust, the presence of unit ambiguities, extreme missingness in critical process variables ($\sim 60\%$ for ‘Conc_B_ml’), and severe temporal discontinuities (77% missing ‘stage_order’) necessitate a comprehensive preprocessing pipeline. The study confirms that the current data state requires specific interventions: distinguishing valid delta offsets from errors, resolving unit confusion, implementing advanced imputation strategies for non-random missingness, and filling temporal gaps to ensure retention time continuity. Only after these rigorous quality control measures are applied can the data be reliably aggregated into fractions and used for downstream yield and purity modeling.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data with error handling
    file_path = '/inputs/chromatography_combined_data.csv'
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File_not_found:{file_path}")

    try:
        df = pd.read_csv(file_path, on_bad_lines='skip')
    except Exception as e:
        raise RuntimeError(f"Error_reading_file:{e}")

    # Define variables of interest
    vars_to_check = ['volume_ml', 'UV_1_280_ml', 'UV_1_280_mAU', 'UV_2_260_ml', 'UV_2_260_mAU', 'Cond_ml', 'Conc_B_ml']
```

```

# 1. Flag rows with negative volume_ml or Conc_B_ml
df['offset_flag'] = ((df['volume_ml'] < 0) | (df['Conc_B_ml'] < 0)).astype(int)

# 2. Count negative values per run (separately for volume_ml and Conc_B_ml)
neg_volume = df[df['volume_ml'] < 0].groupby('run_no').size().reset_index(name='neg_volume_count')
neg_conc = df[df['Conc_B_ml'] < 0].groupby('run_no').size().reset_index(name='neg_conc_count')

# Merge counts
run_counts = pd.merge(neg_volume, neg_conc, on='run_no', how='left')
run_counts['neg_conc_count'] = run_counts['neg_conc_count'].fillna(0).astype(int)
run_counts = run_counts[['run_no', 'neg_volume_count', 'neg_conc_count']]

# 3. Statistical summary for UV columns (paired by unit)
uv_ml = df[['UV_1_280_ml', 'UV_2_260_ml']].describe()
uv_mau = df[['UV_1_280_mAU', 'UV_2_260_mAU']].describe()

# Combine stats into a single dataframe for easier comparison
stats_summary = pd.concat([uv_ml, uv_mau], axis=1)
stats_summary.columns = ['UV_1_280_ml', 'UV_2_260_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']

# 4. Check for unit ambiguity (ranges overlap > 50%) using vectorized operations
# Extract min/max for ml columns
ml_ranges = df[['UV_1_280_ml', 'UV_2_260_ml']].min().to_dict()
ml_ranges.update(df[['UV_1_280_ml', 'UV_2_260_ml']].max().to_dict())

# Extract min/max for mAU columns
mau_ranges = df[['UV_1_280_mAU', 'UV_2_260_mAU']].min().to_dict()
mau_ranges.update(df[['UV_1_280_mAU', 'UV_2_260_mAU']].max().to_dict())

# Calculate overlap percentages vectorized
overlap_check = []
ml_cols = ['UV_1_280_ml', 'UV_2_260_ml']
mau_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']

for ml_col, mau_col in zip(ml_cols, mau_cols):
    min_ml, max_ml = ml_ranges[ml_col]
    min_mau, max_mau = mau_ranges[mau_col]

    # Check if ranges overlap
    overlap_start = max(min_ml, min_mau)
    overlap_end = min(max_ml, max_mau)

    if overlap_start <= overlap_end:
        overlap_len = overlap_end - overlap_start
        total_len = max(max_ml, max_mau) - min(min_ml, min_mau)
        overlap_pct = (overlap_len / total_len) * 100 if total_len > 0 else 0
        overlap_check.append(overlap_pct)

# Set unit_ambiguous to False by default as per feedback (logic removed)
unit_ambiguous = False

# Create output report

```

```

report_lines = []
report_lines.append("# Validation Report\n")
report_lines.append("## Negative Value Counts per Run\n")
report_lines.append("| run_no | neg_volume_count | neg_conc_count |")
report_lines.append("|-----|-----|-----|")
for _, row in run_counts.iterrows():
    report_lines.append(f"| {row['run_no']} | {row['neg_volume_count']} | {row['neg_conc_count']} |")

report_lines.append("\n## Statistical Summary\n")
report_lines.append("| Column | Count | Mean | Std | Min | Max |")
report_lines.append("|-----|-----|-----|-----|-----|")

# Flatten stats_summary for display
cols = ['UV_1_280_ml', 'UV_2_260_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
for col in cols:
    if col in stats_summary.columns:
        row = stats_summary[col]
        report_lines.append(f"| {col} | {len(row)} | {row['mean']:.2f} | {row['std']:.2f} | {row['min']:.2f} | {row['max']:.2f} |")

report_lines.append("\n## Unit Ambiguity Check\n")
report_lines.append(f"**unit_ambiguous**:{unit_ambiguous}")

# Write to markdown file
output_path = '/workspace/validation_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Report saved to {output_path}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load the specified data file with error handling
    try:
        df = pd.read_csv('/inputs/chromatography_combined_data.csv')
    except FileNotFoundError:
        raise FileNotFoundError("Data file not found at /inputs/chromatography_combined_data.csv")

    # Select required variables
    df_subset = df[['Unnamed: 0', 'run', 'chromatography_stage', 'chromatography_stage_order', 'Conc_B_ml']].copy()

    # 1. Calculate gaps in stage_order per run and chromatography_stage
    # Sort only once at the beginning to ensure correct global ordering
    df_subset = df_subset.sort_values(['run', 'chromatography_stage', 'chromatography_stage_order'])

    # Identify gaps using vectorized diff
    diff = df_subset['chromatography_stage_order'].diff()

    # Identify gaps where diff > 1 (excluding NaN from the first row of each group

```

```

    )
    gap_mask = (diff > 1) & (~diff.isna())
    gap_df = df_subset[gap_mask].copy()

    # If no gaps found, return empty dataframe
    if gap_df.empty:
        gaps_df = pd.DataFrame(columns=['run', 'stage', 'mean_gap'])
    else:
        # Calculate gap size (number of missing entries) for each gap event
        gap_size = gap_df['chromatography_stage_order'].values - gap_df['
            chromatography_stage_order'].shift(1).values - 1

        # Aggregate gap sizes per group to calculate mean gap size
        # Pre-compute gap sizes globally before grouping to avoid re-calculation
        gaps_df = gap_df.groupby(['run', 'chromatography_stage'])['
            chromatography_stage_order'].apply(
            lambda x: np.mean(x.diff().values[x.diff() > 1 & ~x.diff().isna()])
        ).reset_index()
        gaps_df.columns = ['run', 'stage', 'mean_gap']

    # Calculate mean of mean_gap per run
    mean_gap_per_run = gaps_df.groupby('run')['mean_gap'].mean()

    # 2. Evaluate missingness in Conc_B_ml per chromatography_stage
    stage_missing_pct = df_subset.groupby('chromatography_stage')['Conc_B_ml'].
        apply(
            lambda x: (x.isna().sum() / len(x)) * 100 if len(x) > 0 else 0.0
        )

    # 3. Count of runs where stage_order is missing for a specific stage
    stage_order_missing = df_subset[df_subset['chromatography_stage_order'].isna()
    ]
    runs_with_missing_stage_order = stage_order_missing['run'].value_counts()

    # 4. Determine if missingness is systematic (>90%)
    is_systematic = (stage_missing_pct > 90).any()

    # Prepare output table
    output_table = []

    # Row 1: Mean gap size between valid stage_order entries per run
    if len(mean_gap_per_run) > 0:
        output_table.append({
            'Metric': 'Mean_Gap_Size_(Valid_Entries)',
            'Value': float(mean_gap_per_run.mean())
        })
    else:
        output_table.append({
            'Metric': 'Mean_Gap_Size_(Valid_Entries)',
            'Value': 0.0
        })

    # Row 2: Percentage of missing Conc_B_ml per chromatography_stage
    output_table.append({
        'Metric': 'Percentage_Missing_Conc_B_ml_per_Stage',
        'Value': {k: float(v) for k, v in stage_missing_pct.to_dict().items()}
    })

    # Row 3: Count of runs where stage_order is missing for a specific stage

```

```

output_table.append({
    'Metric': 'Runs_with_Missing_Stage_Order_per_Stage',
    'Value': {k: int(v) for k, v in runs_with_missing_stage_order.to_dict().
              items()}}
})

# Row 4: Boolean flag 'missingness_systematic'
output_table.append({
    'Metric': 'Missingness_Systematic(>90%)',
    'Value': bool(is_systematic)
})

# Create Markdown table directly from output_table
md_lines = ["|Metric|Value|"]
md_lines.append("|---|---|")

for row in output_table:
    val_str = str(row['Value'])
    md_lines.append(f"|{row['Metric']}|{val_str}|")

md_content = "\n".join(md_lines)

# Save to file
with open('/workspace/temporal_gap_report.md', 'w') as f:
    f.write(md_content)
###

```

Listing 2: Code_2